利用1D边缘提取方法计算亚像素边缘点、利用最新二乘直线拟合计算边缘直线，再计算两条直线的交点坐标。

步骤:

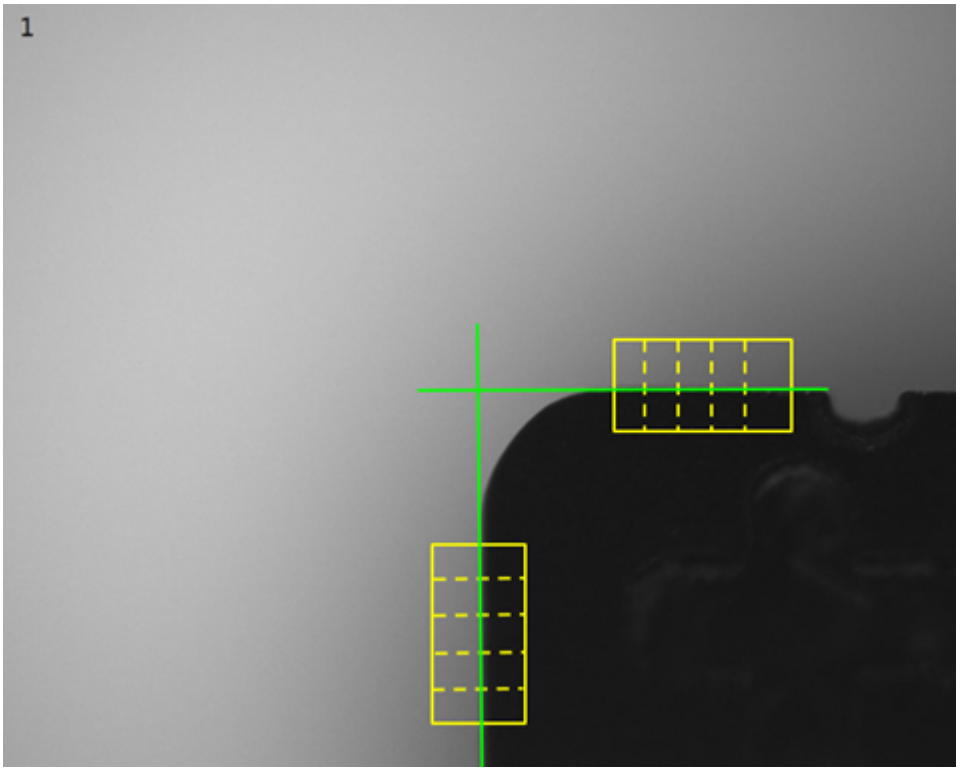
设置矩形ROI区域;

设置采样线间隔;

在每条采样线上计算亚像素边缘点;

利用亚像素边缘点拟合直线，并计算直线的交点。

要求:以word或pdf形式提交算法的细节步骤，给出代码以及结果数据。



算法步骤

首先读取原始图像，并将彩色图像转换为灰度图像。为了减小图像噪声对边缘梯度计算的影响，对灰度图像进行高斯滤波处理。

然后根据图像中黑色目标的位置，分别设置上边缘 ROI 和左边缘 ROI。对于上边缘区域，使用若干条竖直采样线；对于左边缘区域，使用若干条水平采样线。采样线按照固定间隔排列，本实验中采样间隔设置为 10 像素。

在每一条采样线上，读取灰度值形成一维灰度序列，并计算相邻像素之间的灰度梯度。由于黑色目标与白色背景之间存在明显灰度突变，梯度幅值最大的区域即为边缘所在位置。为了获得比整数像素更高的定位精度，在最大梯度点附近利用相邻梯度比例进行线性插值，从而得到亚像素级边缘点坐标。

获得上边缘和左边缘的全部亚像素边缘点后，分别采用总最小二乘法进行直线拟合。拟合直线统一表示为：

$$ax + by + c = 0$$

其中 a、b、c 为拟合得到的直线参数。最后将上边缘直线和左边缘直线联立求解，得到两条直线的交点坐标。交点计算公式为：

$$D = a_1 b_2 - a_2 b_1$$

代码

```
#include <opencv2/opencv.hpp>
#include <iostream>
#include <vector>
#include <cmath>
using namespace cv;
using namespace std;

struct LineABC {
    double a, b, c; // ax + by + c = 0
};

Point2d subpixelEdge1D(const Mat& gray, Point p1, Point p2) {
    LineIterator it(gray, p1, p2, 8);
    vector<double> v;
    vector<Point2d> pts;

    for (int i = 0; i < it.count; i++, ++it) {
        Point pt = it.pos();
        v.push_back(gray.at<uchar>(pt));
        pts.push_back(Point2d(pt.x, pt.y));
    }

    int best = 1;
    double maxGrad = 0;
    for (int i = 1; i < (int)v.size() - 1; i++) {
        double g = fabs(v[i + 1] - v[i - 1]);
        if (g > maxGrad) {
            maxGrad = g;
```

```
            best = i;
        }
    }

    double g1 = fabs(v[best] - v[best - 1]);
    double g2 = fabs(v[best + 1] - v[best]);
    double offset = 0.0;
    if (g1 + g2 > 1e-6) {
        offset = g2 / (g1 + g2);
    }

    Point2d q = pts[best] * (1.0 - offset) + pts[best + 1] * offset;
    return q;
}

LineABC fitLineTLS(const vector<Point2d>& pts) {
    Point2d mean(0, 0);
    for (auto& p : pts) mean += p;
    mean.x /= pts.size();
    mean.y /= pts.size();

    double Sxx = 0, Sxy = 0, Syy = 0;
    for (auto& p : pts) {
        double x = p.x - mean.x;
        double y = p.y - mean.y;
        Sxx += x * x;
        Sxy += x * y;
        Syy += y * y;
    }

    Mat M = (Mat_<double>(2, 2) << Sxx, Sxy, Sxy, Syy);
    Mat eigVal, eigVec;
    eigen(M, eigVal, eigVec);

    // 最小特征值对应法向量
    double a = eigVec.at<double>(1, 0);
    double b = eigVec.at<double>(1, 1);
    double c = -(a * mean.x + b * mean.y);

    double norm = sqrt(a * a + b * b);
    return {a / norm, b / norm, c / norm};
}

Point2d intersectLines(LineABC l1, LineABC l2) {
    double D = l1.a * l2.b - l2.a * l1.b;
    double Dx = l1.b * l2.c - l2.b * l1.c;
    double Dy = l1.c * l2.a - l2.c * l1.a;
    return Point2d(Dx / D, Dy / D);
```

```
}

int main() {
    vector<string> files = {
        "作业7_1.jpg",
        "作业7_2.jpg",
        "作业7_3.jpg",
        "作业7_4.jpg"
    };

    for (auto& file : files) {
        Mat img = imread(file);
        if (img.empty()) {
            cout << "Cannot read: " << file << endl;
            continue;
        }

        Mat gray;
        cvtColor(img, gray, COLOR_BGR2GRAY);
        GaussianBlur(gray, gray, Size(5, 5), 1.0);

        vector<Point2d> topPts;
        vector<Point2d> leftPts;

        // 上边缘：竖直采样线，从白到黑
        for (int x = 850; x <= 1300; x += 10) {
            Point2d p = subpixelEdge1D(gray, Point(x, 560), Point(x,
720));
            topPts.push_back(p);
        }

        // 左边缘：水平采样线，从白到黑
        for (int y = 730; y <= 1000; y += 10) {
            Point2d p = subpixelEdge1D(gray, Point(650, y), Point(850,
y));
            leftPts.push_back(p);
        }

        LineABC topLine = fitLineTLS(topPts);
        LineABC leftLine = fitLineTLS(leftPts);
        Point2d cross = intersectLines(topLine, leftLine);

        cout << "Image: " << file << endl;
        cout << "Top line:  " << topLine.a << "x + " << topLine.b << "y +
" << topLine.c << " = 0" << endl;
        cout << "Left line: " << leftLine.a << "x + " << leftLine.b << "y
+ " << leftLine.c << " = 0" << endl;
```

```
        cout << "Intersection: (" << cross.x << ", " << cross.y << ")" <<
endl;
        cout << "Top points: " << topPts.size() << ", Left points: " <<
leftPts.size() << endl;
        cout << "---------------------------------" << endl;

        Mat show = img.clone();
        for (auto& p : topPts) circle(show, p, 2, Scalar(0, 0, 255), -1);
        for (auto& p : leftPts) circle(show, p, 2, Scalar(255, 0, 0),
-1);
        circle(show, cross, 5, Scalar(0, 255, 0), -1);

        imwrite("result_" + file, show);
    }

    return 0;
}
```

结果

```
PS C:\Onedrive\Coding\Opencv> .\main.exe
Image: 7_1.jpg
Top line:  -0.0274965x + 0.999622y + -604.381 = 0
Left line: -0.947461x + -0.319871y + 975.807 = 0
Intersection: (818.198, 627.115)
Top points: 46, Left points: 28
---------------------------------
Image: 7_2.jpg
Top line:  0.0691364x + 0.997607y + -694.423 = 0
Left line: -0.919274x + 0.393618y + 322.253 = 0
Intersection: (629.914, 652.435)
Top points: 46, Left points: 28
---------------------------------
Image: 7_3.jpg
Top line:  0.0591463x + 0.998249y + -689.112 = 0
Left line: -0.998992x + 0.0448857y + 670.024 = 0
Intersection: (699.853, 648.854)
Top points: 46, Left points: 28
---------------------------------
Image: 7_4.jpg
Top line:  -0.0170239x + 0.999855y + -593.581 = 0
Left line: -0.996879x + 0.0789422y + 655.158 = 0
Intersection: (705.171, 605.673)
Top points: 46, Left points: 28
---------------------------------
```